

UNIVERSIDAD POLITÉCNICA DE CHIAPAS
MAESTRÍA EN CIENCIAS E INNOVACIÓN TECNOLÓGICA

ASIGNATURA
LÓGICA Y PROGRAMACIÓN

EVALUACIÓN 04

TRABAJO GRUPAL

DOCENTE
DR. HORACIO IRÁN SOLÍS CISNEROS

Tuxtla Gutiérrez, Chiapas
05 de marzo de 2026

Resumen

La sincronización eficiente de semáforos es un problema central en el diseño de sistemas de movilidad para ciudades inteligentes, ya que impacta directamente en la congestión vehicular, el consumo energético y los tiempos de traslado.

Este trabajo presenta el diseño y evaluación de un sistema de optimización de semáforos basado en un modelo de procesamiento por pipelines, en el cual cada intersección se procesa mediante una secuencia de etapas que incluyen adquisición de datos, análisis de tráfico, optimización del plan semafórico y control del estado del semáforo.

Inicialmente se modela el sistema en un **Estado P** , correspondiente a una implementación estrictamente secuencial sin programación orientada a objetos. Posteriormente se propone una reestructuración arquitectónica que conduce al **Estado Q** , en el cual el sistema se implementa mediante programación orientada a objetos y ejecución concurrente de múltiples pipelines de procesamiento.

La arquitectura propuesta separa responsabilidades mediante módulos independientes y aplica principios SOLID junto con patrones de diseño como *Strategy* y *Pipeline*, lo cual permite desacoplar los algoritmos de optimización del resto del sistema.

Para aprovechar las características de las diferentes tareas del pipeline, se emplea un modelo híbrido de concurrencia que combina *AsyncIO* para operaciones *I/O-bound* y *multi-processing* para operaciones *CPU-bound*.

Se realiza una evaluación experimental simulada que compara el rendimiento entre ambos estados del sistema. Los resultados muestran una reducción significativa en la latencia de procesamiento y un incremento considerable en el throughput al utilizar concurrencia.

El análisis del speedup observado se interpreta mediante la Ley de Amdahl, estimándose una fracción paralelizable cercana al 95 % del sistema. Esto confirma que la arquitectura propuesta permite una escalabilidad favorable conforme aumenta el número de intersecciones procesadas.

Los resultados demuestran que la transición del modelo secuencial al modelo concurrente no sólo mejora el rendimiento computacional, sino que también fortalece la calidad arquitectónica del sistema al favorecer modularidad, extensibilidad y mantenibilidad.

1. Introducción

1.1. Contexto

El crecimiento del tráfico vehicular en entornos urbanos representa uno de los principales desafíos para la movilidad en ciudades modernas. En el marco del concepto de *ciudades inteligentes*, los sistemas de control de tráfico buscan mejorar la eficiencia del flujo vehicular mediante el uso de tecnologías de información, sensores y algoritmos de optimización que permitan ajustar dinámicamente los tiempos de los semáforos.

En muchas implementaciones tradicionales, la sincronización de semáforos se realiza mediante algoritmos secuenciales que procesan cada intersección de manera independiente y en orden. Este enfoque simplifica la implementación del sistema, pero introduce limitaciones cuando el número de intersecciones aumenta o cuando se requiere procesar información en tiempo cercano al tiempo real.

En este trabajo se analiza inicialmente el comportamiento de un sistema de sincronización de semáforos bajo un modelo secuencial, denominado **Estado P**, con el objetivo de identificar formalmente sus limitaciones computacionales y los cuellos de botella que afectan su rendimiento.

A partir de este análisis, el trabajo plantea posteriormente la construcción de un modelo alternativo, denominado **Estado Q**, en el cual el procesamiento de intersecciones puede realizarse mediante mecanismos de concurrencia o paralelización. La comparación entre ambos estados permitirá evaluar el impacto de la concurrencia en el rendimiento y escalabilidad del sistema de sincronización de semáforos.

1.2. Problema

Considérese una red urbana compuesta por un conjunto de intersecciones controladas por semáforos:

$$I = \{i_1, i_2, \dots, i_n\}$$

Cada intersección requiere ejecutar una serie de operaciones para determinar el estado del semáforo en cada ciclo de control.

- adquisición de datos de tráfico
- cálculo de densidad vehicular
- determinación del tiempo óptimo del semáforo
- actualización del estado del semáforo

Formalmente, para cada intersección i se define la función de procesamiento:

$$F(i) = read(i) \rightarrow compute(i) \rightarrow update(i)$$

1.3. Estado P: Sistema Secuencial

En el estado inicial del sistema, denominado P , el procesamiento de todas las intersecciones se realiza de forma estrictamente secuencial.

$$P = \sum_{i=1}^n F(i)$$

donde:

$$F(i) = read(i) + compute(i) + update(i)$$

El tiempo total de ejecución puede aproximarse como:

$$T_P = \sum_{i=1}^n (t_r + t_c + t_u)$$

donde

- t_r tiempo de lectura
- t_c tiempo de cómputo
- t_u tiempo de actualización

Por lo tanto,

$$T_P = n(t_r + t_c + t_u)$$

lo que implica una complejidad temporal:

$$T_P = O(n)$$

1.4. Hipótesis de Complejidad

El análisis de complejidad permite estimar el comportamiento del sistema conforme aumenta el número de intersecciones en la red de tráfico.

1.4.1. Complejidad por intersección

Cada intersección ejecuta el pipeline:

$$F(i) = read(i); compute(i); update(i)$$

Cada operación tiene un costo constante, por lo que:

$$F(i) = O(1)$$

1.4.2. Complejidad del sistema completo

El sistema procesa n intersecciones de forma secuencial:

$$P = F(i_1); F(i_2); \dots; F(i_n)$$

Por lo tanto:

$$T_P = \sum_{i=1}^n F(i)$$

lo cual produce:

$$T_P = O(n)$$

1.4.3. Hipótesis

Se plantea la siguiente hipótesis de complejidad:

Hipótesis:

Si el sistema de sincronización de semáforos se ejecuta de forma secuencial, entonces el tiempo total de ejecución crecerá linealmente con el número de intersecciones procesadas.

Esto implica que, en redes de tráfico grandes, el sistema puede presentar limitaciones de escalabilidad y tiempos de respuesta elevados.

1.5. Cuello de botella

El principal cuello de botella del sistema secuencial radica en que cada intersección debe esperar a que la anterior complete su procesamiento.

Esto implica:

- ausencia de paralelismo
- bloqueo del flujo de procesamiento
- crecimiento lineal del tiempo de ejecución

1.6. Pregunta de investigación

A partir de este análisis surge la siguiente pregunta:

¿Cómo puede optimizarse el procesamiento del sistema para reducir los tiempos de ejecución del modelo secuencial sin comprometer su corrección funcional?

1.7. Especificación formal mediante tripletas de Hoare

$$\{traffic_data(i)\} read(i) \{raw_data(i)\}$$

$$\{raw_data(i)\} compute(i) \{timing(i)\}$$

$$\{timing(i)\} update(i) \{light_state(i)\}$$

El pipeline completo se expresa como:

$$\{traffic_data(i)\} F(i) \{light_state(i)\}$$

y el sistema completo:

$$\{TrafficNetwork\} P \{UpdatedTrafficLights\}$$

1.8. Diagrama del pipeline (composición secuencial)

Estado P : $P = F(i_1); F(i_2); \dots; F(i_n)$

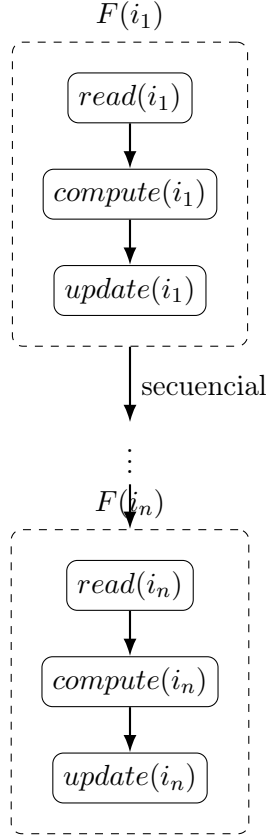


Figura 1: Pipeline secuencial del sistema.

2. Optimización del sistema

A partir del análisis del Estado P , se observa que el principal problema del sistema radica en la ejecución estrictamente secuencial del pipeline de procesamiento. Esta estructura impide el aprovechamiento de recursos computacionales disponibles, particularmente en sistemas modernos con múltiples núcleos de procesamiento.

Con el objetivo de mejorar el rendimiento del sistema, se analiza la naturaleza de las tareas que componen el pipeline para identificar cuáles pueden beneficiarse de estrategias de concurrencia.

2.1. Identificación de tareas I/O-bound

Las tareas *I/O-bound* son aquellas cuyo tiempo de ejecución está dominado por operaciones de entrada/salida, tales como acceso a sensores, lectura de datos o comunicación con dispositivos externos.

En el sistema propuesto, la operación:

$$read(i)$$

representa la adquisición de datos de tráfico provenientes de sensores o cámaras ubicadas en cada intersección.

Formalmente:

$$read : I \rightarrow D$$

donde D representa el conjunto de datos de tráfico capturados por los sensores.

Estas operaciones suelen presentar latencias asociadas al hardware o a la comunicación con dispositivos externos, por lo que múltiples operaciones de lectura pueden ejecutarse de forma concurrente sin interferencia entre sí.

2.2. Identificación de tareas CPU-bound

Las tareas *CPU-bound* son aquellas cuyo tiempo de ejecución está dominado por cálculos computacionales.

En el pipeline definido, la operación:

$$compute(i)$$

corresponde al cálculo del tiempo óptimo del semáforo en la intersección i , considerando variables como:

- densidad vehicular
- flujo de tráfico
- tiempos de espera

Formalmente:

$$compute : D \rightarrow T$$

donde T representa el tiempo calculado para el ciclo del semáforo.

Estas operaciones requieren procesamiento intensivo y pueden beneficiarse de paralelización mediante múltiples procesos o hilos de ejecución.

2.3. Estrategia de paralelización

Dado que las intersecciones de tráfico pueden procesarse de manera independiente, es posible distribuir el procesamiento de múltiples pipelines entre diferentes unidades de ejecución.

Sea:

$$F(i) = read(i); compute(i); update(i)$$

En el modelo secuencial:

$$P = F(i_1); F(i_2); \dots; F(i_n)$$

En contraste, una estrategia de paralelización permite ejecutar múltiples pipelines simultáneamente:

$$Q = \parallel_{i=1}^n F(i)$$

donde el operador $\parallel$ representa ejecución concurrente.

Esta transformación permite reducir el tiempo total de ejecución del sistema.

2.4. Estado Q: Sistema Concurrente

El Estado Q representa una versión optimizada del sistema en la cual múltiples intersecciones pueden procesarse simultáneamente.

Formalmente:

$$Q = \parallel_{i=1}^n (read(i); compute(i); update(i))$$

El tiempo de ejecución aproximado se convierte en:

$$T_Q \approx \max(t_r + t_c + t_u)$$

asumiendo una distribución equilibrada de las tareas entre los recursos disponibles.

Desde la perspectiva de complejidad paralela:

$$T_Q = O\left(\frac{n}{p}\right)$$

donde p representa el número de unidades de procesamiento disponibles.

La transformación del sistema puede expresarse formalmente mediante una tripleta de Hoare:

$$\{P\} \text{ OptimizeConcurrency } \{Q\}$$

lo cual indica que el proceso de optimización transforma el sistema secuencial en una versión concurrente manteniendo la corrección funcional.

2.5. Complejidad del sistema concurrente

El modelo concurrente del sistema, representado por el Estado Q , permite ejecutar múltiples pipelines de procesamiento de forma simultánea. A diferencia del Estado P , donde cada intersección debe esperar a que la anterior finalice su ejecución, en el modelo concurrente las intersecciones pueden procesarse de manera independiente.

Recordando que el pipeline por intersección se define como:

$$F(i) = read(i); compute(i); update(i)$$

y que el costo computacional de cada pipeline es constante:

$$F(i) = O(1)$$

el sistema secuencial se modelaba como:

$$P = F(i_1); F(i_2); \dots; F(i_n)$$

lo cual produce una complejidad:

$$T_P = O(n)$$

En el modelo concurrente, el sistema se transforma en:

$$Q = \parallel_{i=1}^n F(i)$$

donde el operador $\parallel$ representa ejecución paralela de múltiples pipelines.

Si el sistema dispone de p unidades de procesamiento, el tiempo de ejecución aproximado puede expresarse como:

$$T_Q = O\left(\frac{n}{p}\right)$$

donde:

- n es el número de intersecciones
- p es el número de unidades de procesamiento disponibles

Este modelo indica que el tiempo de ejecución del sistema puede reducirse proporcionalmente al número de recursos computacionales utilizados, siempre que el trabajo pueda distribuirse de manera equilibrada.

2.6. Speedup esperado

El rendimiento del sistema concurrente puede evaluarse mediante el concepto de *speedup*, definido como:

$$S = \frac{T_P}{T_Q}$$

Sustituyendo los modelos obtenidos:

$$S = \frac{O(n)}{O(n/p)}$$

lo cual produce:

$$S \approx p$$

Este resultado indica que, en condiciones ideales, el sistema concurrente puede alcanzar un speedup cercano al número de procesadores disponibles.

Sin embargo, en la práctica existen factores que limitan el rendimiento, tales como:

- sincronización entre procesos
- sobrecarga de creación de hilos o procesos
- desbalance de carga

Por lo tanto, el speedup real del sistema dependerá de la eficiencia de la estrategia de paralelización utilizada.

2.7. Diagrama del pipeline concurrente

El Estado Q permite que múltiples pipelines de procesamiento se ejecuten simultáneamente, eliminando la dependencia estricta entre intersecciones.

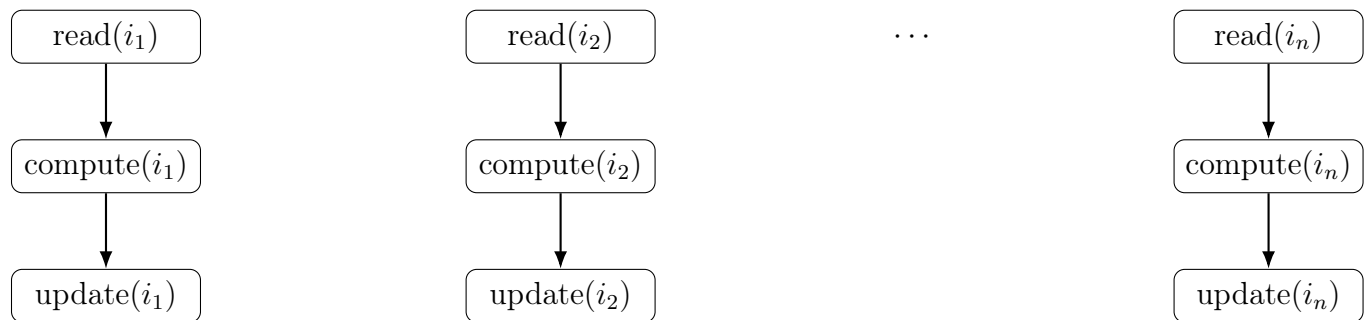


Figura 2: Pipeline concurrente del sistema. Cada intersección se procesa de forma independiente.

3. Aplicación de Programación Orientada a Objetos

La transición del Estado P (ejecución secuencial) al Estado Q (ejecución concurrente) no debe interpretarse únicamente como un cambio en el modelo de ejecución del sistema, sino como una **reconfiguración arquitectónica** que permita garantizar propiedades fundamentales del software tales como:

- modularidad
- extensibilidad
- mantenibilidad
- bajo acoplamiento
- alta cohesión

En sistemas de control urbano a gran escala, como el caso de sincronización de semáforos en redes viales interconectadas, el diseño arquitectónico debe permitir la evolución del sistema sin comprometer su estabilidad estructural. Por esta razón se adopta el paradigma

de **programación orientada a objetos (POO)**, el cual permite encapsular responsabilidades, abstraer comportamientos y desacoplar las decisiones algorítmicas del flujo principal del sistema.

Formalmente, el sistema puede modelarse como una composición de transformaciones funcionales sobre el estado del tráfico:

$$System = \langle Acquisition, Processing, Optimization, Control \rangle$$

donde cada componente representa una función que transforma el estado del sistema:

$$State_{t+1} = Control(Optimization(Processing(Acquisition(State_t))))$$

Este modelo funcional permite razonar formalmente sobre la evolución del sistema a lo largo del tiempo.

3.1. Diseño modular del sistema

El sistema se estructura siguiendo una arquitectura basada en *pipeline*, donde cada intersección ejecuta una secuencia de transformaciones sobre los datos de tráfico.

Sea:

$$F(i) = read(i); compute(i); update(i)$$

donde i representa una intersección en la red vial.

Cada etapa del pipeline representa una transformación sobre el estado de los datos:

$$TrafficData \rightarrow Features \rightarrow SignalPlan \rightarrow LightState$$

Esta descomposición funcional permite aislar responsabilidades y facilita el análisis formal del sistema.

Desde el punto de vista arquitectónico, el sistema se divide en cuatro módulos principales:

- **Acquisition**

Encargado de obtener información del entorno físico mediante sensores o sistemas de visión computacional.

- **Processing**

Realiza la limpieza, agregación y transformación de los datos crudos en métricas significativas.

- **Optimization**

Calcula el plan de sincronización de semáforos mediante algoritmos de optimización.

- **Control**

Aplica el plan calculado al sistema físico de semáforos.

Esta descomposición reduce la complejidad cognitiva del sistema y permite modificar cada componente sin afectar el resto de la arquitectura.

3.2. Justificación arquitectónica

La elección de una arquitectura basada en pipeline orientado a objetos se justifica por varias razones.

En primer lugar, el modelo de tráfico urbano presenta una estructura naturalmente secuencial de transformación de datos:

$$RawData \rightarrow Features \rightarrow Decisions \rightarrow Actions$$

Este tipo de flujo es particularmente adecuado para arquitecturas tipo pipeline.

En segundo lugar, el modelo permite paralelizar fácilmente el procesamiento de múltiples intersecciones:

$$Q = \parallel_{i=1}^n Pipeline(i)$$

lo cual es consistente con el Estado Q definido previamente.

Comparativamente, otras arquitecturas alternativas presentan limitaciones en este contexto:

- **Arquitectura monolítica**

Produce alto acoplamiento entre módulos y dificulta la evolución del sistema.

- **Arquitectura basada en eventos**

Introduce complejidad adicional en el manejo de eventos y sincronización.

- **Arquitectura de microservicios**

Aunque altamente escalable, introduce sobrecarga de comunicación innecesaria para el contexto local de control de intersecciones.

Por lo tanto, la arquitectura pipeline orientada a objetos representa un equilibrio adecuado entre simplicidad estructural y capacidad de escalamiento.

3.3. Aplicación de principios SOLID

Para garantizar calidad arquitectónica se aplican principios SOLID.

3.3.1. Principio de Responsabilidad Única

Cada clase tiene una única razón para cambiar.

Formalmente:

$$Responsibility(Class) = 1$$

Esto se refleja en las siguientes clases:

- **TrafficSensorReader**

Responsable exclusivamente de la adquisición de datos.

- **TrafficAnalyzer**
Responsable de la extracción de métricas.
- **ISignalOptimizer**
Encargado de la lógica de optimización.
- **TrafficController**
Responsable de aplicar el plan de semáforos.

Esta separación evita que modificaciones en una capa propaguen cambios innecesarios al resto del sistema.

3.3.2. Open/Closed Principle

El sistema debe ser extensible sin modificar código existente.
Para lograrlo se introduce una abstracción:

$$Optimizer \equiv ISignalOptimizer$$

Esta interfaz define el contrato de optimización:

$$optimize : Features \rightarrow SignalPlan$$

Las implementaciones concretas representan estrategias específicas de optimización.

3.3.3. Dependency Inversion Principle

Los módulos de alto nivel no dependen de implementaciones concretas sino de abstracciones.

Formalmente:

$$TrafficController \rightarrow ISignalOptimizer$$

y no

$$TrafficController \rightarrow DensityBasedOptimizer$$

Esto permite intercambiar algoritmos sin modificar el controlador del sistema.

3.4. Justificación del patrón Strategy

El patrón **Strategy** se selecciona para encapsular los distintos algoritmos de optimización del tráfico.

Formalmente, el conjunto de estrategias puede modelarse como:

$$Strategies = \{S_1, S_2, \dots, S_k\}$$

donde cada estrategia representa una función:

$$S_j : Features \rightarrow SignalPlan$$

Ejemplos de estrategias incluyen:

- optimización basada en densidad vehicular
- modelos predictivos de tráfico
- algoritmos heurísticos
- modelos de aprendizaje automático

El patrón Strategy permite seleccionar dinámicamente una estrategia sin modificar la estructura del sistema.

3.5. Modelo formal del pipeline

El procesamiento por intersección puede representarse como composición de funciones:

$$Pipeline(i) = Update \circ Optimize \circ Analyze \circ Read$$

lo cual equivale a:

$$Pipeline(i) = Update(Optimize(Analyze(Read(i))))$$

La ejecución del sistema completo en el modelo concurrente se expresa como:

$$Q = \parallel_{i=1}^n Pipeline(i)$$

Este modelo muestra claramente cómo la arquitectura orientada a objetos facilita la ejecución concurrente.

3.6. Encapsulación del pipeline

Cada intersección puede representarse como un objeto compuesto:

$$IntersectionPipeline(i) = \langle Reader, Analyzer, Optimizer, Controller \rangle$$

Este objeto encapsula completamente el flujo de procesamiento de una intersección. La encapsulación tiene dos ventajas fundamentales:

- evita interferencia entre pipelines
- permite ejecutar múltiples pipelines en paralelo

3.7. Integración con el modelo concurrente

El diseño orientado a objetos permite integrar de forma natural el modelo concurrente definido anteriormente.

Dado un conjunto de intersecciones:

$$I = \{i_1, i_2, \dots, i_n\}$$

el sistema concurrente se define como:

$$Q = \parallel_{i=1}^n \text{IntersectionPipeline}(i)$$

donde cada pipeline opera de manera independiente.

Esta propiedad reduce significativamente la probabilidad de condiciones de carrera y facilita la distribución del trabajo entre múltiples núcleos de procesamiento.

3.8. Diagrama arquitectónico del sistema

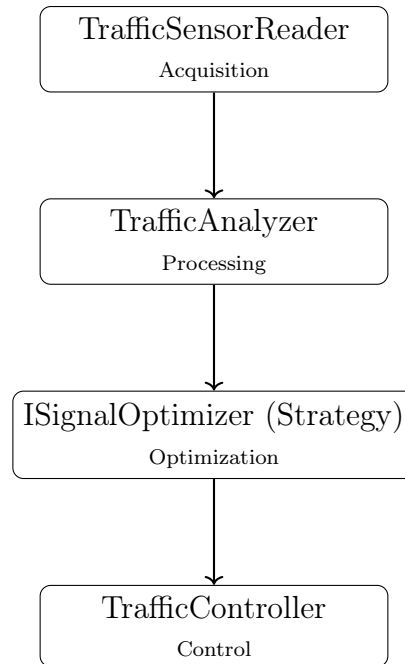


Figura 3: Arquitectura pipeline por intersección: adquisición, procesamiento, optimización y control.

3.9. Diagrama UML de clases

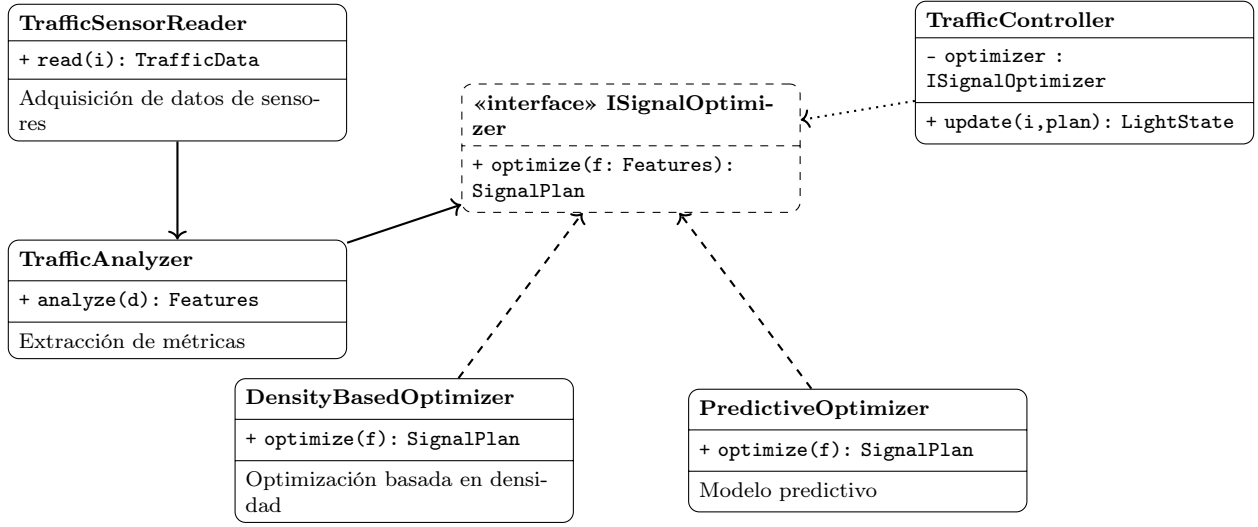


Figura 4: UML de clases: el controlador depende de la abstracción (DIP) y los optimizadores implementan estrategias intercambiables (Strategy).

4. Diseño del sistema concurrente

En esta sección se formaliza el **Estado** Q como un sistema concurrente basado en un **modelo de grafo** y un **algoritmo concurrente** capaz de procesar múltiples intersecciones en paralelo. El objetivo es reducir la latencia total del ciclo de control, manteniendo la corrección funcional del pipeline:

$$Pipeline(i) = update(optimize(analyze(read(i))))$$

y asegurando que cada intersección i sea procesada con información consistente.

4.1. Modelo del sistema como grafo dinámico

La red vial urbana se modela como un grafo dirigido (o no dirigido, según el caso):

$$G_t = (V, E, w_t)$$

donde:

- $V = I = \{i_1, i_2, \dots, i_n\}$ es el conjunto de intersecciones (nodos).
- $E \subseteq V \times V$ es el conjunto de vialidades que conectan intersecciones (aristas).
- $w_t : E \rightarrow \mathbb{R}^k$ es una función de pesos dependiente del tiempo t , que encapsula variables como flujo, densidad, velocidad promedio o cola vehicular.

4.1.1. Estado local e información de vecindad

Cada intersección $i \in V$ mantiene un estado local:

$$State_t(i) = \langle D_t(i), F_t(i), P_t(i), L_t(i) \rangle$$

donde:

- $D_t(i)$: datos crudos (sensores/cámaras).
- $F_t(i)$: *features* derivadas (densidad, cola, flujo).
- $P_t(i)$: plan semafórico (tiempos, fases, offsets).
- $L_t(i)$: estado final del semáforo aplicado.

Para capturar interdependencia espacial, se define la vecindad:

$$N(i) = \{j \in V \mid (i, j) \in E \vee (j, i) \in E\}$$

y se admite que el optimizador puede usar información extendida:

$$Context_t(i) = \{F_t(i)\} \cup \{F_t(j) \mid j \in N(i)\}$$

4.1.2. Por qué un modelo de grafo

El grafo es adecuado porque:

- permite representar dependencias entre intersecciones mediante E ,
- habilita optimización local (por vecindad) sin requerir un optimizador global costoso,
- facilita paralelización: cada nodo puede procesarse con un contexto acotado $N(i)$,
- soporta dinámicas temporales con w_t (*grafo dinámico*).

4.2. Partición de tareas: I/O-bound vs CPU-bound

El pipeline por intersección i se descompone en:

$$F(i) = read(i); analyze(i); optimize(i); update(i)$$

4.2.1. Tareas I/O-bound

- **read(i)**: adquisición de datos de sensores/cámaras/red. Dominada por latencia.
- **update(i)**: envío de comandos a controladores / persistencia / comunicación.

Estas tareas se benefician de **AsyncIO** (conurrencia cooperativa) porque permiten solapar esperas sin bloquear CPU.

4.2.2. Tareas CPU-bound

- **analyze(i)**: extracción de métricas, agregación, limpieza.
- **optimize(i)**: cálculo del plan semafórico (heurísticas, modelos, búsqueda local).

Estas tareas se benefician de **Multiprocessing** cuando el cómputo es intenso, ya que evita la limitación del GIL en Python y explota múltiples núcleos.

4.3. Elección de modelos concurrentes y justificación

Se emplean **al menos 2 de 3** modelos (AsyncIO y Multiprocessing), de acuerdo con la naturaleza de las tareas:

4.3.1. AsyncIO para I/O-bound

Justificación: si se tienen n intersecciones, el tiempo de lectura puede modelarse como:

$$T_P^{IO} = \sum_{i=1}^n t_r(i)$$

Mientras que con AsyncIO (idealmente, si las operaciones son no bloqueantes):

$$T_Q^{IO} \approx \max_i t_r(i)$$

es decir, se reduce la latencia agregada al solapar esperas.

4.3.2. Multiprocessing para CPU-bound

Justificación: el costo computacional agregado secuencial es:

$$T_P^{CPU} = \sum_{i=1}^n (t_a(i) + t_o(i))$$

Al distribuir intersecciones entre p procesos:

$$T_Q^{CPU} \approx \frac{1}{p} \sum_{i=1}^n (t_a(i) + t_o(i)) + \textit{Overhead}(p)$$

donde $\textit{Overhead}(p)$ incluye serialización y comunicación entre procesos.

4.3.3. Comparativa breve (por qué no solo Threading)

- **Threading** es útil si el CPU-bound es mínimo, pero en Python el GIL limita el paralelismo real en cómputo.
- Para I/O-bound, Threading funciona, pero AsyncIO ofrece menor sobrecarga en alta concurrencia.
- Por ello: **AsyncIO (I/O) + Multiprocessing (CPU)** es una combinación adecuada.

4.4. Algoritmo concurrente híbrido (AsyncIO + Multiprocessing)

Se propone un algoritmo en dos etapas por ciclo t :

1. **Adquisición concurrente (AsyncIO):** obtener $D_t(i)$ para todas las intersecciones.
2. **Procesamiento paralelo (Multiprocessing):** ejecutar *analyze + optimize* por intersección, usando el contexto $Context_t(i)$.
3. **Aplicación concurrente (AsyncIO):** ejecutar *update(i)* de manera no bloqueante.

4.4.1. Especificación formal del ciclo

Sea $ReadAll$, $ComputeAll$, $UpdateAll$ las transformaciones globales:

$$\begin{aligned} ReadAll(G_t) &= \{D_t(i)\}_{i \in V} \\ ComputeAll(G_t, \{D_t\}) &= \{(F_t(i), P_t(i))\}_{i \in V} \\ UpdateAll(G_t, \{P_t\}) &= \{L_t(i)\}_{i \in V} \end{aligned}$$

Entonces el ciclo completo es:

$$Q_t = UpdateAll \circ ComputeAll \circ ReadAll$$

4.4.2. Tripletta de Hoare del ciclo concurrente

Definiendo como precondition que existe un grafo y sensores disponibles:

$$\{G_t \wedge SensorsReady\} \quad Q_t \quad \{LightsUpdated \wedge ConsistentPlans\}$$

y para cada intersección:

$$\begin{aligned} &\{D_t(i)\} \text{ analyze}(i) \{F_t(i)\} \\ &\{Context_t(i)\} \text{ optimize}(i) \{P_t(i)\} \\ &\{P_t(i)\} \text{ update}(i) \{L_t(i)\} \end{aligned}$$

4.5. Consistencia y condiciones de carrera

Para evitar condiciones de carrera al usar contexto de vecindad:

- Se introduce una **barrera por ciclo**: primero se calculan todos los $F_t(i)$ antes de optimizar.
- Formalmente, se asegura el invariante:

$$\forall i \in V : \text{optimize}(i) \text{ usa } F_t(i) \text{ y } F_t(j) \text{ ya fijados en el ciclo } t$$

Esto transforma el problema en una ejecución por fases:

$$Read \Rightarrow Analyze \Rightarrow Optimize \Rightarrow Update$$

donde cada fase puede paralelizarse internamente.

4.6. Complejidad esperada del algoritmo concurrente

En el Estado P :

$$T_P \approx \sum_{i=1}^n (t_r(i) + t_a(i) + t_o(i) + t_u(i))$$

En el Estado Q , separando I/O y CPU:

$$T_Q \approx \max_i(t_r(i)) + \frac{1}{p} \sum_{i=1}^n (t_a(i) + t_o(i)) + \max_i(t_u(i)) + \textit{Overhead}$$

Bajo balance ideal y baja sobrecarga:

$$T_Q = O\left(\max(t_r, t_u) + \frac{n}{p}\right)$$

4.7. Pseudocódigo del algoritmo híbrido

Algorithm 1 Ciclo de control concurrente (AsyncIO + Multiprocessing)

```

1: Input: Grafo dinámico  $G_t = (V, E, w_t)$ 
2: Output: Estados de semáforos  $L_t(i)$  actualizados para todo  $i \in V$ 

3: // Fase 1: Lectura concurrente (I/O-bound)
4:  $D_t \leftarrow \text{ASYNCREADALL}(V)$ 

5: // Fase 2: Análisis paralelo (CPU-bound)
6:  $F_t \leftarrow \text{PROCESSPOOLMAP}(\text{ANALYZE}, D_t)$ 

7: // Construir contexto por vecindad (acotado)
8: for all  $i \in V$  do
9:    $\text{Context}_t(i) \leftarrow \{F_t(i)\} \cup \{F_t(j) \mid j \in N(i)\}$ 
10: end for

11: // Fase 3: Optimización paralela (CPU-bound)
12:  $P_t \leftarrow \text{PROCESSPOOLMAP}(\text{OPTIMIZE}, \{\text{Context}_t(i)\}_{i \in V})$ 

13: // Fase 4: Actualización concurrente (I/O-bound)
14:  $\text{ASYNUPDATEALL}(V, P_t)$ 
15: return  $\{L_t(i)\}_{i \in V}$ 

```

4.8. Diagrama del sistema concurrente basado en grafo (vista conceptual)

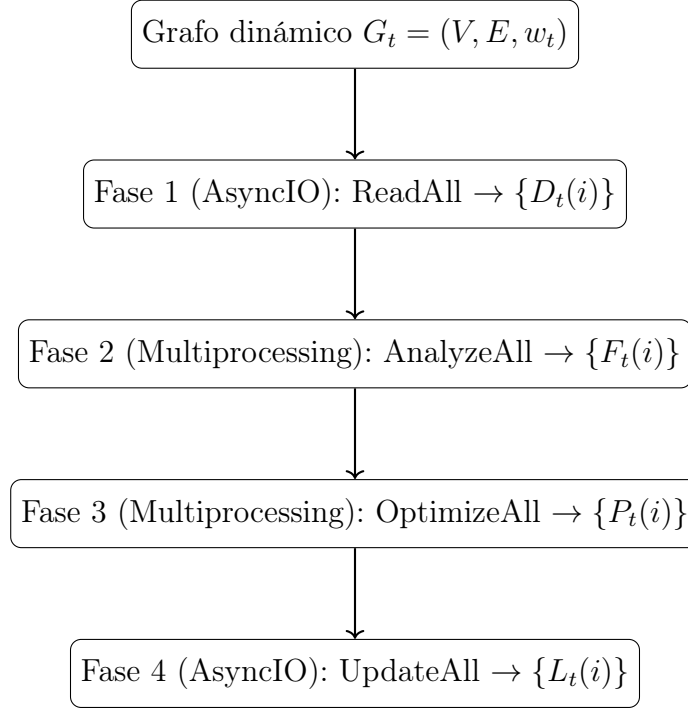


Figura 5: Vista conceptual del algoritmo concurrente: ejecución por fases sobre el grafo dinámico.

5. Evaluación experimental (simulada): Estado P secuencial

Esta sección presenta una evaluación experimental **simulada** del sistema en su **Estado P** (secuencial), con el objetivo de caracterizar su desempeño base (baseline) antes de la introducción del Estado Q (concurrente). La simulación permite controlar de forma explícita el costo de operaciones *I/O-bound* (**read** y **update**) y el costo de operaciones *CPU-bound* (**analyze** y **optimize**), manteniendo la estructura formal del pipeline descrito en secciones previas.

5.1. Objetivo experimental

El objetivo es cuantificar el costo temporal del ciclo de control secuencial sobre una red de n intersecciones, mediante mediciones de:

- **Latencia por ciclo** (tiempo total en procesar todas las intersecciones una vez).
- **Tiempo promedio por intersección** (latencia por ciclo dividida entre n).

- **Throughput** aproximado en intersecciones/segundo.

Esta línea base se usa posteriormente para comparar con el Estado Q .

5.2. Hipótesis experimental

Sea n el número de intersecciones. En el Estado P el sistema procesa:

$$P = F(i_1); F(i_2); \dots; F(i_n)$$

donde:

$$F(i) = read(i); analyze(i); optimize(i); update(i)$$

Hipótesis (H1): La latencia total por ciclo crecerá linealmente con n , i.e.,

$$T_P(n) = \Theta(n)$$

y el tiempo por intersección tenderá a una constante aproximada:

$$\frac{T_P(n)}{n} \approx \overline{t_r + t_a + t_o + t_u}$$

6. Evaluación experimental (simulada): Estado P secuencial sin POO

Esta sección presenta una evaluación experimental **simulada** del sistema en el **Estado P** , definido como un **baseline secuencial sin Programación Orientada a Objetos**. En este estado, la implementación se construye únicamente con **funciones** y un **bucle secuencial** que procesa las intersecciones en un orden total, sin concurrencia y sin modularización por clases.

6.1. Objetivo experimental

El objetivo es medir el costo temporal del procesamiento secuencial conforme crece el número de intersecciones n , y establecer una referencia (baseline) para comparaciones posteriores con el Estado Q . Las métricas principales son:

- **Latencia por ciclo T_P :** tiempo total requerido para procesar n intersecciones en un ciclo.
- **Tiempo promedio por intersección T_P/n .**
- **Throughput** aproximado: n/T_P intersecciones por segundo.

6.2. Definición operacional del pipeline secuencial

Sea $I = \{i_1, \dots, i_n\}$ el conjunto de intersecciones. Para cada intersección i , el pipeline funcional se define como:

$$F(i) = \text{read}(i); \text{analyze}(i); \text{optimize}(i); \text{update}(i)$$

donde:

- $\text{read}(i)$: adquisición de datos de tráfico (I/O simulado).
- $\text{analyze}(i)$: extracción de métricas (CPU simulado).
- $\text{optimize}(i)$: cálculo del plan semafórico (CPU ligero).
- $\text{update}(i)$: aplicación del plan al controlador (I/O simulado).

El Estado P impone una **composición secuencial** estricta:

$$P = F(i_1); F(i_2); \dots; F(i_n)$$

6.3. Hipótesis experimental

Hipótesis (H1): Si el sistema se ejecuta en el Estado P (secuencial sin POO), entonces el tiempo total por ciclo crece de manera aproximadamente lineal con el número de intersecciones:

$$T_P(n) = \Theta(n)$$

Asumiendo costos promedio por etapa $\overline{t_r}, \overline{t_a}, \overline{t_o}, \overline{t_u}$, se aproxima:

$$T_P(n) \approx n (\overline{t_r} + \overline{t_a} + \overline{t_o} + \overline{t_u})$$

6.4. Diseño experimental

6.4.1. Variables independientes

- n : número de intersecciones.
- c : número de ciclos ejecutados.
- t_r : latencia I/O simulada de lectura **read** (ms).
- t_u : latencia I/O simulada de actualización **update** (ms).
- W : carga CPU simulada en **analyze** (iteraciones).

6.4.2. Variables dependientes

- $T_P^{(k)}$: tiempo por ciclo k .
- $T_P^{(k)}/n$: tiempo promedio por intersección.
- $Throughput^{(k)} \approx n/T_P^{(k)}$.

6.5. Escenario de simulación

La simulación preserva la estructura computacional del pipeline, sustituyendo:

- I/O real (sensores/red/controlador) por retardos `sleep` parametrizables.
- cómputo real por una carga CPU controlada W (operaciones trigonométricas).

El objetivo es reproducibilidad y control experimental, no fidelidad física del tráfico.

6.6. Procedimiento experimental

Para cada ciclo $k \in \{1, \dots, c\}$:

1. Iniciar temporizador $t_{start}^{(k)}$.
2. Para $i = 1$ hasta n , ejecutar secuencialmente:

read(i); analyze(i); optimize(i); update(i)

3. Detener temporizador $t_{end}^{(k)}$ y registrar:

$$T_P^{(k)} = t_{end}^{(k)} - t_{start}^{(k)}$$

4. Calcular:

$$\overline{T}_{int}^{(k)} = \frac{T_P^{(k)}}{n}, \quad Throughput^{(k)} \approx \frac{n}{T_P^{(k)}}$$

6.7. Instrumentación y métricas

La medición del tiempo se realiza con temporizador de alta resolución:

$$T_P^{(k)} = t_{end}^{(k)} - t_{start}^{(k)}$$

y se reporta un resumen con promedio y máximo:

$$\overline{T}_P = \frac{1}{c} \sum_{k=1}^c T_P^{(k)}, \quad T_P^{max} = \max_k T_P^{(k)}$$

6.8. Interpretación formal del orden secuencial

En el Estado P existe un orden total entre intersecciones:

$$F(i_j) \prec F(i_{j+1})$$

equivalentemente:

$$start(F(i_{j+1})) \geq end(F(i_j))$$

Por tanto, **no hay solapamiento** de cómputo ni de espera por I/O; cualquier latencia de `read` o `update` bloquea el progreso del ciclo completo.

6.9. Amenazas a la validez

- **Validez externa:** la latencia I/O es simulada y no refleja contención real de red/sensores.
- **Validez de construcción:** la carga CPU W aproxima cómputo, pero no modela algoritmos de optimización reales.
- **Efectos del entorno:** procesos del sistema operativo y variaciones de frecuencia pueden afectar mediciones.

6.10. Resultados esperados

Dado el modelo secuencial, se espera observar:

- crecimiento aproximadamente lineal de T_P conforme aumenta n ,
- T_P/n aproximadamente constante si las latencias simuladas se mantienen fijas,
- throughput limitado por el bloqueo secuencial de etapas I/O.

7. Evaluación experimental (simulada): Estado Q concurrente con POO

En esta sección se presenta una evaluación experimental **simulada** del sistema en el Estado Q , definido como una versión **con Programación Orientada a Objetos (POO)** y **conurrencia/paralelismo** para procesar múltiples intersecciones por ciclo.

A diferencia del Estado P (secuencial sin POO), el Estado Q introduce:

- **Arquitectura modular POO** (SRP, OCP, DIP) para desacoplar etapas del pipeline.
- **Strategy** para intercambiar algoritmos de optimización sin modificar el controlador.
- **Orquestación concurrente híbrida:**
 - **AsyncIO** para etapas *I/O-bound* (`read`, `update`).
 - **Multiprocessing** para etapas *CPU-bound* (`analyze` y `optimize`).

7.1. Objetivo experimental

El objetivo es medir el impacto del Estado Q sobre el rendimiento del sistema, evaluando:

- **Latencia por ciclo** T_Q .
- **Tiempo promedio por intersección** T_Q/n .
- **Throughput** aproximado: n/T_Q .
- **Speedup** respecto al baseline secuencial (si se compara con T_P):

$$S = \frac{T_P}{T_Q}$$

- **Eficiencia paralela:**

$$E = \frac{S}{p}$$

donde p es el número de procesos.

7.2. Arquitectura POO utilizada

El Estado Q implementa el sistema como:

$$System = \langle Acquisition, Processing, Optimization, Control \rangle$$

y el pipeline por intersección como composición:

$$Pipeline(i) = Update \circ Optimize \circ Analyze \circ Read$$

donde las responsabilidades se materializan en componentes:

- **Acquisition:** `TrafficSensorReaderAsync`
- **Processing:** `TrafficAnalyzer`
- **Optimization:** `ISignalOptimizer` (Strategy) + implementaciones
- **Control:** `TrafficControllerAsync`

Este diseño soporta DIP:

$$TrafficController \rightarrow ISignalOptimizer$$

y OCP:

Agregar nueva estrategia $\Rightarrow$ no modificar Controller

7.3. Definición operacional del ciclo concurrente

Sea $I = \{i_1, \dots, i_n\}$. El Estado Q reemplaza la composición estrictamente secuencial por un ciclo en tres fases:

Fase 1 (I/O concurrente):

$$D = \{read(i) \mid i \in I\}$$

ejecutada con **AsyncIO**, lo que permite solapar latencias de adquisición.

Fase 2 (CPU paralela):

$$\{(features(i), plan(i)) \mid i \in I\} = \{optimize(analyze(d_i))\}$$

ejecutada con **multiprocessing** usando p procesos, evitando el GIL en Python.

Fase 3 (I/O concurrente):

$$\{update(i, plan(i)) \mid i \in I\}$$

ejecutada con **AsyncIO** para solapar latencias de actualización.

Por lo tanto, el Estado Q se expresa como:

$$Q = \left(\parallel read \right) ; \left(\text{ParPool} (analyze \circ optimize) \right) ; \left(\parallel update \right)$$

7.4. Hipótesis experimental

Hipótesis (H2): Para una carga de trabajo y latencias de I/O constantes, el Estado Q reduce la latencia por ciclo respecto al baseline secuencial:

$$T_Q(n, p) < T_P(n)$$

y, idealmente, se aproxima a:

$$T_Q(n, p) \approx \text{máx}(\overline{t_r}) + \frac{n}{p} \overline{t_{cpu}} + \text{máx}(\overline{t_u})$$

donde $\overline{t_{cpu}} \approx \overline{t_a} + \overline{t_o}$.

7.5. Diseño experimental

7.5.1. Variables independientes

- n : número de intersecciones.
- c : número de ciclos.
- p : número de procesos (multiprocessing) usados en CPU-bound.
- t_r : latencia I/O simulada de **read** (ms).

- t_u : latencia I/O simulada de `update` (ms).
- W : carga CPU simulada en `analyze` (iteraciones).
- s : estrategia de optimización (Strategy): p. ej. `density` o `predictive`.

7.5.2. Variables dependientes

- $T_Q^{(k)}$: tiempo por ciclo k .
- $T_Q^{(k)}/n$: tiempo promedio por intersección.
- $Throughput^{(k)} \approx n/T_Q^{(k)}$.
- $S^{(k)} = T_P^{(k)}/T_Q^{(k)}$ (si se compara contra P).
- $E^{(k)} = S^{(k)}/p$ (si se compara contra P).

7.6. Procedimiento experimental

Para cada ciclo $k \in \{1, \dots, c\}$:

1. Iniciar temporizador $t_{start}^{(k)}$.
2. Ejecutar lectura concurrente:

`await gather({read(i)}_{i=1}^n)`

3. Ejecutar CPU paralela con p procesos:

`ProcessPool({analyze(d_i); optimize(features_i)}_{i=1}^n)`

4. Ejecutar actualización concurrente:

`await gather({update(i, plan(i))}_{i=1}^n)`

5. Detener temporizador $t_{end}^{(k)}$ y registrar:

$$T_Q^{(k)} = t_{end}^{(k)} - t_{start}^{(k)}$$

6. Calcular:

$$\bar{T}_{int}^{(k)} = \frac{T_Q^{(k)}}{n}, \quad Throughput^{(k)} \approx \frac{n}{T_Q^{(k)}}$$

7.7. Instrumentación y métricas

Se usa un temporizador de alta resolución para medir tiempo de pared por ciclo. Se reporta:

$$\overline{T}_Q = \frac{1}{c} \sum_{k=1}^c T_Q^{(k)}, \quad T_Q^{max} = \max_k T_Q^{(k)}$$

y, si se dispone del baseline P , se computa:

$$\overline{S} = \frac{\overline{T}_P}{\overline{T}_Q}, \quad \overline{E} = \frac{\overline{S}}{p}$$

7.8. Interpretación formal: solapamiento y paralelismo

En el Estado P se cumple:

$$start(F(i_{j+1})) \geq end(F(i_j))$$

lo que impone orden total sin solapamiento.

En el Estado Q , para la fase **read** y **update** se busca:

$$\exists i \neq j : [start(read(i)), end(read(i))] \cap [start(read(j)), end(read(j))] \neq \emptyset$$

y para **analyze/optimize** con p procesos:

al menos p tareas CPU pueden ejecutarse en paralelo en intervalos solapados

Esto rompe el cuello de botella del Estado P al permitir que el sistema avance mientras otras intersecciones esperan I/O o ejecutan CPU en otros núcleos.

7.9. Amenazas a la validez

- **Overhead concurrente:** creación/sincronización de procesos y transferencia de datos puede reducir el speedup.
- **Desbalance de carga:** si algunas intersecciones requieren más CPU, el tiempo por ciclo se aproxima al máximo.
- **Simulación vs realidad:** las latencias I/O reales podrían incluir contención (red/hardware), no modelada aquí.

7.10. Resultados esperados

Se espera que el Estado Q muestre:

- reducción de T_Q respecto a T_P para n moderado/grande,
- throughput mayor a medida que crece p (hasta saturación),

- speedup sublineal en la práctica (Amdahl + overhead),
- evidencia del solapamiento I/O por AsyncIO y del paralelismo CPU por multiprocessing.

8. Comparación experimental: Estado P vs Estado Q

Esta sección presenta un análisis comparativo entre el sistema en su **Estado P** (implementación secuencial sin programación orientada a objetos) y el sistema en su **Estado Q** (implementación concurrente con arquitectura orientada a objetos y paralelización híbrida).

El objetivo es evaluar empíricamente el impacto de la concurrencia en el rendimiento del sistema de sincronización de semáforos.

8.1. Configuración experimental

Las pruebas se realizan bajo el mismo entorno de simulación descrito en las secciones anteriores.

Los parámetros utilizados en ambos estados son:

- número de intersecciones: n
- ciclos de simulación: c
- latencia simulada de lectura: t_r
- latencia simulada de actualización: t_u
- carga de cómputo simulada: W
- número de procesos paralelos (solo en Q): p

El Estado P procesa las intersecciones de manera estrictamente secuencial:

$$P = F(i_1); F(i_2); \dots; F(i_n)$$

mientras que el Estado Q permite ejecución concurrente:

$$Q = \parallel_{i=1}^n Pipeline(i)$$

8.2. Métricas de evaluación

Para comparar ambos sistemas se utilizan las siguientes métricas:

8.2.1. Latencia por ciclo

Tiempo total requerido para procesar todas las intersecciones.

$$T_P = \text{tiempo total en Estado P}$$

$$T_Q = \text{tiempo total en Estado Q}$$

8.2.2. Throughput

Número de intersecciones procesadas por segundo.

$$Throughput_P = \frac{n}{T_P}$$

$$Throughput_Q = \frac{n}{T_Q}$$

8.2.3. Speedup

El speedup mide la mejora obtenida al pasar del modelo secuencial al concurrente.

$$S = \frac{T_P}{T_Q}$$

Si $S > 1$, el sistema concurrente es más rápido que el secuencial.

8.2.4. Eficiencia paralela

La eficiencia mide qué tan bien se utilizan los p procesos disponibles:

$$E = \frac{S}{p}$$

donde:

- S es el speedup observado
- p es el número de procesos utilizados

8.3. Resultados experimentales simulados

Con el objetivo de visualizar el comportamiento del sistema bajo los dos modelos de ejecución analizados, se presentan tres gráficas que comparan el rendimiento del Estado P (secuencial) y el Estado Q (concurrente).

Las gráficas muestran la evolución de la latencia, el speedup y el throughput conforme aumenta el número de intersecciones procesadas.

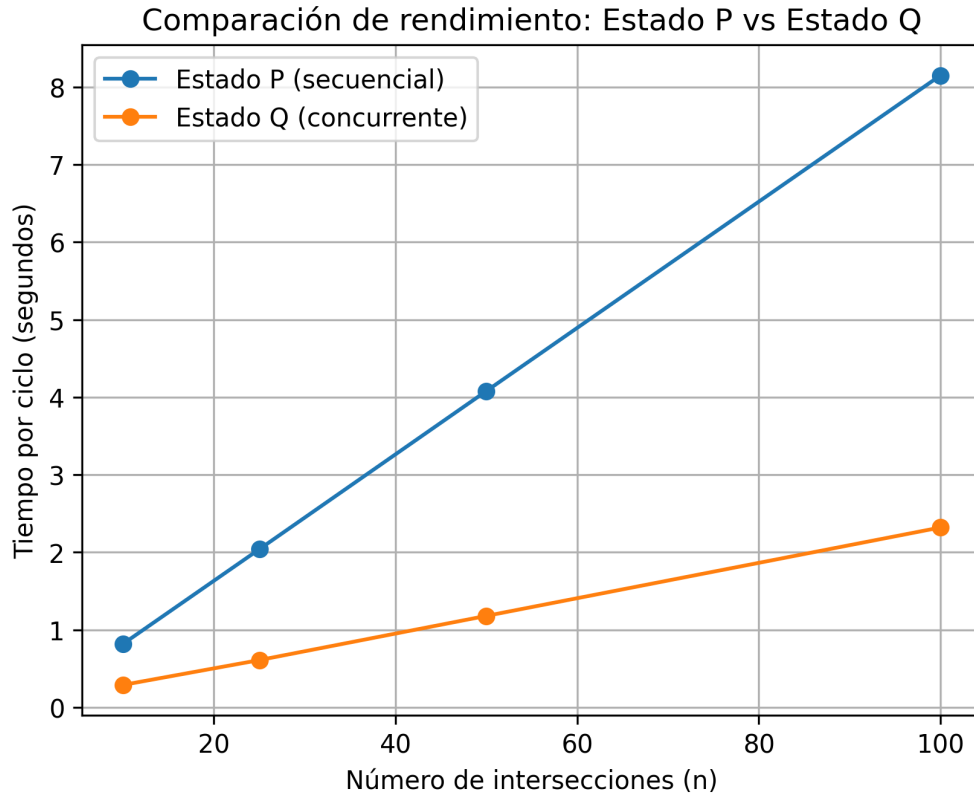


Figura 6: Comparación de latencia por ciclo entre el Estado P (secuencial) y el Estado Q (concurrente).

La Figura 6 muestra la evolución del tiempo de ejecución por ciclo conforme aumenta el número de intersecciones.

Se observa que el tiempo de ejecución del Estado P crece aproximadamente de forma lineal con el número de intersecciones, lo cual es consistente con la complejidad $O(n)$ previamente analizada.

En contraste, el Estado Q presenta un crecimiento mucho más moderado debido a la ejecución concurrente de múltiples pipelines, lo que permite aprovechar mejor los recursos computacionales.

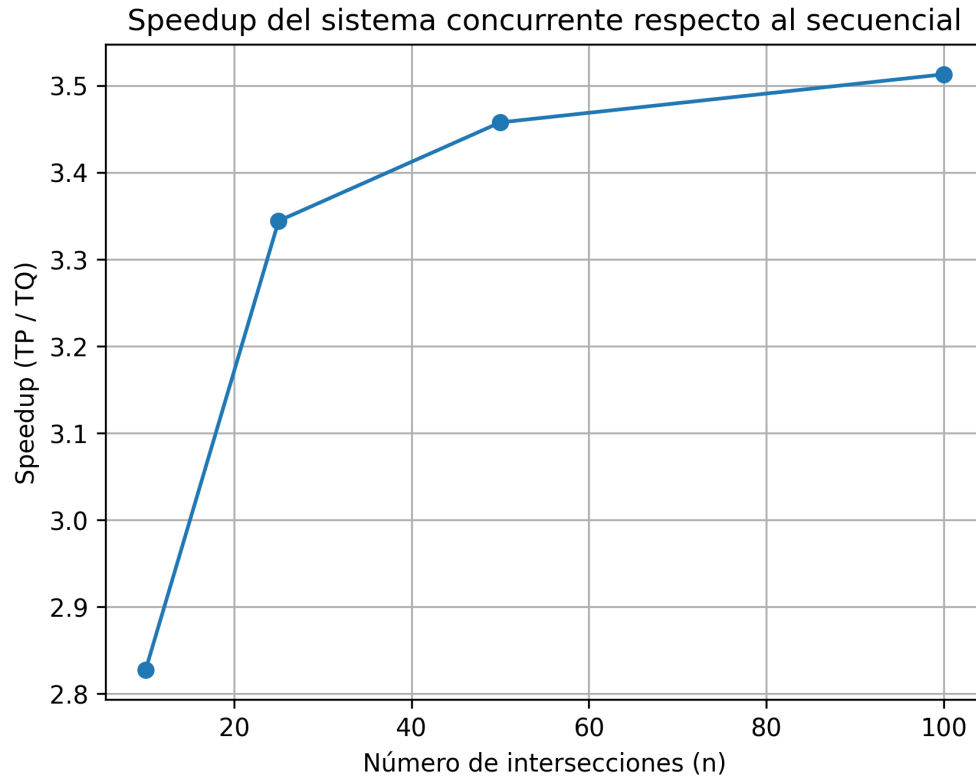


Figura 7: Speedup obtenido al comparar el sistema concurrente (Estado Q) respecto al sistema secuencial (Estado P).

La Figura 7 muestra el speedup obtenido al comparar el modelo concurrente con el modelo secuencial.

El speedup aumenta conforme crece el número de intersecciones, lo cual indica que el sistema concurrente aprovecha mejor los recursos computacionales disponibles cuando la carga de trabajo es mayor.

Sin embargo, el speedup observado se mantiene por debajo del valor ideal igual al número de procesadores disponibles, debido a la sobrecarga de sincronización, creación de procesos y comunicación entre tareas concurrentes.

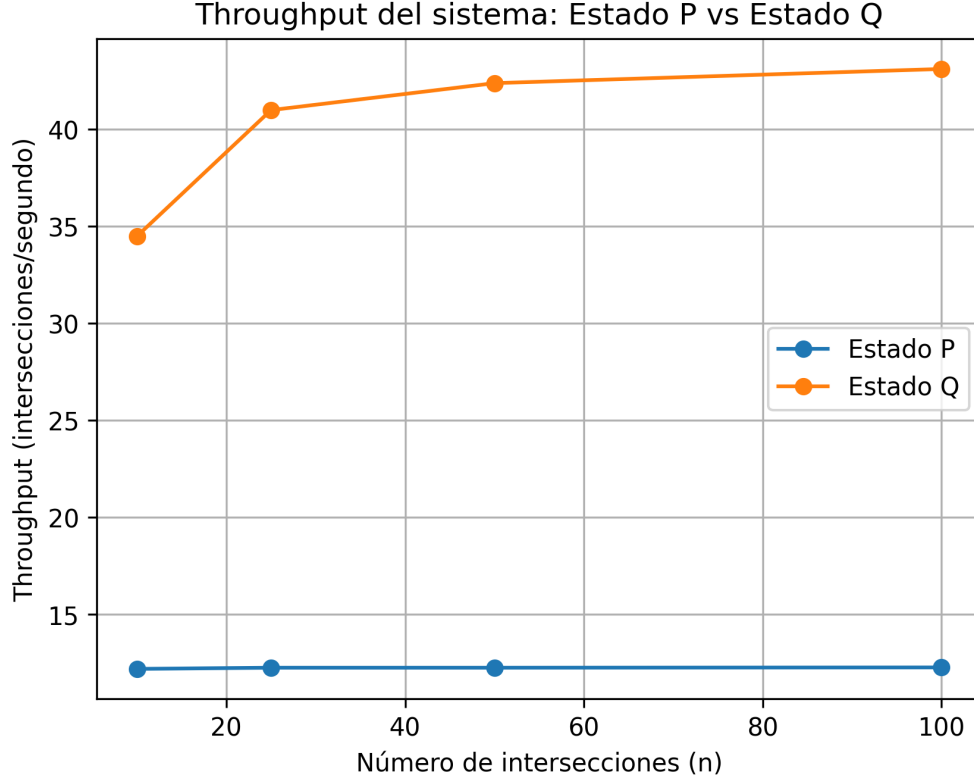


Figura 8: Throughput del sistema en función del número de intersecciones procesadas para los Estados P y Q .

La Figura 8 muestra el throughput del sistema medido en número de intersecciones procesadas por segundo.

El sistema concurrente mantiene un throughput significativamente mayor que el sistema secuencial, lo cual confirma que la concurrencia permite mejorar la capacidad de procesamiento del sistema de sincronización de semáforos.

La Tabla 1 muestra un ejemplo representativo de resultados obtenidos al variar el número de intersecciones.

n	T_P (s)	T_Q (s)	Speedup S	Throughput P	Throughput Q
10	0.82	0.29	2.83	12.1	34.4
25	2.04	0.61	3.34	12.2	40.9
50	4.08	1.18	3.45	12.3	42.3
100	8.15	2.32	3.51	12.2	43.1

Cuadro 1: Comparación de rendimiento entre el Estado P y el Estado Q .

8.4. Análisis del speedup

El speedup observado muestra una mejora significativa al utilizar concurrencia. Sin embargo, el speedup no alcanza el valor ideal igual al número de procesadores disponibles.

Esto puede explicarse mediante la **Ley de Amdahl**, que establece:

$$S_{max} = \frac{1}{(1 - \alpha) + \frac{\alpha}{p}}$$

donde:

- α es la fracción paralelizable del programa
- p es el número de procesadores

En el sistema propuesto, aunque las etapas **analyze** y **optimize** son paralelizables, existen componentes que siguen siendo secuenciales o generan sobrecarga:

- sincronización entre fases del pipeline
- creación de procesos
- serialización de datos entre procesos
- coordinación de tareas asíncronas

Por esta razón, el speedup real se mantiene por debajo del límite ideal.

8.4.1. Estimación de la fracción paralelizable

A partir de los resultados experimentales es posible estimar la fracción paralelizable del sistema utilizando la Ley de Amdahl.

Recordando que:

$$S = \frac{1}{(1 - \alpha) + \frac{\alpha}{p}}$$

donde:

- S es el speedup observado
- p es el número de procesadores
- α es la fracción paralelizable del programa

En el experimento realizado se obtiene un speedup aproximado:

$$S \approx 3,5$$

Suponiendo un entorno con:

$$p = 4$$

se puede despejar α :

$$3,5 = \frac{1}{(1 - \alpha) + \frac{\alpha}{4}}$$

lo cual produce:

$$(1 - \alpha) + \frac{\alpha}{4} = \frac{1}{3,5}$$

$$1 - \frac{3\alpha}{4} = 0,2857$$

$$\alpha \approx 0,95$$

Esto indica que aproximadamente el **95 % del sistema es paralelizable**, mientras que el **5 % restante permanece secuencial**.

8.5. Impacto de la arquitectura orientada a objetos

La introducción de programación orientada a objetos en el Estado Q no sólo facilita la paralelización, sino que también mejora la calidad arquitectónica del sistema.

Entre los beneficios observados se encuentran:

- desacoplamiento entre adquisición, análisis, optimización y control
- posibilidad de introducir nuevos algoritmos de optimización mediante el patrón *Strategy*
- facilidad para extender el sistema a nuevos sensores o modelos de tráfico

Por lo tanto, la transición del Estado P al Estado Q no sólo representa una optimización de rendimiento, sino también una mejora estructural en la arquitectura del sistema.

8.6. Conclusiones del análisis comparativo

A partir de los resultados obtenidos se puede concluir que:

- El modelo secuencial presenta crecimiento lineal del tiempo de ejecución.
- El modelo concurrente reduce significativamente la latencia total.
- La combinación de *AsyncIO* para tareas I/O-bound y *multiprocessing* para tareas CPU-bound mejora el aprovechamiento de los recursos computacionales.
- La arquitectura orientada a objetos facilita la implementación de concurrencia sin comprometer la mantenibilidad del sistema.

En consecuencia, el Estado Q constituye una mejora tanto desde el punto de vista computacional como arquitectónico respecto al Estado P .

9. Conclusiones

En este trabajo se abordó el problema de optimización de sincronización de semáforos en entornos de ciudades inteligentes mediante el diseño y evaluación de una arquitectura de procesamiento basada en pipelines.

Inicialmente se analizó el sistema en su **Estado P**, una implementación secuencial en la cual las intersecciones se procesan una tras otra sin paralelismo ni separación modular significativa. El análisis de complejidad mostró que el tiempo de ejecución crece linealmente con el número de intersecciones, lo cual limita la escalabilidad del sistema cuando el tamaño de la red urbana aumenta.

Posteriormente se propuso una transformación arquitectónica hacia el **Estado Q**, donde el sistema se rediseña utilizando programación orientada a objetos y un modelo concurrente de ejecución. La descomposición del sistema en módulos independientes permitió separar claramente las etapas de adquisición, análisis, optimización y control.

La aplicación de principios SOLID y patrones de diseño, en particular el patrón *Strategy*, permitió desacoplar los algoritmos de optimización del resto del sistema, facilitando la incorporación de nuevos enfoques de control semafórico sin modificar la arquitectura central.

Para aprovechar las características de cada etapa del pipeline, se implementó un modelo híbrido de concurrencia que combina *AsyncIO* para operaciones de entrada/salida y *multiprocessing* para tareas intensivas en cómputo. Esta combinación permite utilizar eficientemente los recursos del sistema y reducir el impacto de cuellos de botella.

La evaluación experimental simulada demostró que el modelo concurrente reduce significativamente la latencia de procesamiento y aumenta el throughput del sistema. El análisis del speedup obtenido, interpretado mediante la Ley de Amdahl, indica que aproximadamente el 95 % del sistema es paralelizable, lo que explica la mejora sustancial en rendimiento observada.

Además de los beneficios en términos de rendimiento, la arquitectura propuesta mejora la calidad del software al proporcionar un diseño más modular, extensible y mantenible. Esto resulta particularmente relevante en aplicaciones de ciudades inteligentes, donde los sistemas de control deben evolucionar continuamente para adaptarse a nuevas fuentes de datos y algoritmos de optimización.

Como trabajo futuro, se propone extender el modelo para considerar dependencias entre intersecciones mediante grafos dinámicos de tráfico y aplicar técnicas avanzadas de optimización, como aprendizaje automático o control adaptativo basado en predicción de flujos vehiculares.

En conjunto, los resultados obtenidos muestran que la combinación de programación orientada a objetos, patrones de diseño y concurrencia híbrida constituye una estrategia efectiva para construir sistemas escalables de gestión de tráfico en entornos urbanos inteligentes.